PROJET · BASES DE DONNÉES DU BIG DATA · M2

Rift Stats

Les statistiques de **League of Legends** avec trois bases qui ont chacune un rôle : MongoDB, Redis et PostgreSQL + pgvector.

538	300	173	16.22
parties en base (MongoDB)	joueurs Challenger (Redis)	champions (PostgreSQL)	patch du jeu analysé

Le besoin

1 **Quels champions gagnent en ce moment ?**

taux de victoire par champion et par rôle

2 **Qui sont les meilleurs joueurs du serveur ?**

classement par points de ligue (LP)

3 **Où en suis-je, moi ?**

son propre profil, chargé à la demande

4 **Quel champion ressemble à celui que je joue ?**

similarité entre champions

LES MÉTRIQUES DU TABLEAU DE BORD

- **Top 10 des champions** par taux de victoire, au moins 20 parties
  agrégation MongoDB
- **Top 10 des joueurs** par LP
  ensemble trié Redis
- **5 champions similaires**
  recherche vectorielle pgvector

2 · SOURCES RÉELLES

Les données

Source	Ce qu'on récupère	Volume
API Riot · League-V4	le classement Challenger du serveur EUW	300 joueurs
API Riot · Account-V1	le Riot ID (Pseudo#TAG) des 30 premiers	30 comptes
API Riot · Match-V5	leurs 20 dernières parties classées solo/duo	510 parties
Data Dragon 16.22.1	fiches des champions et des objets (sans clé)	173 + 863

LA CONTRAINTE : LE QUOTA RIOT

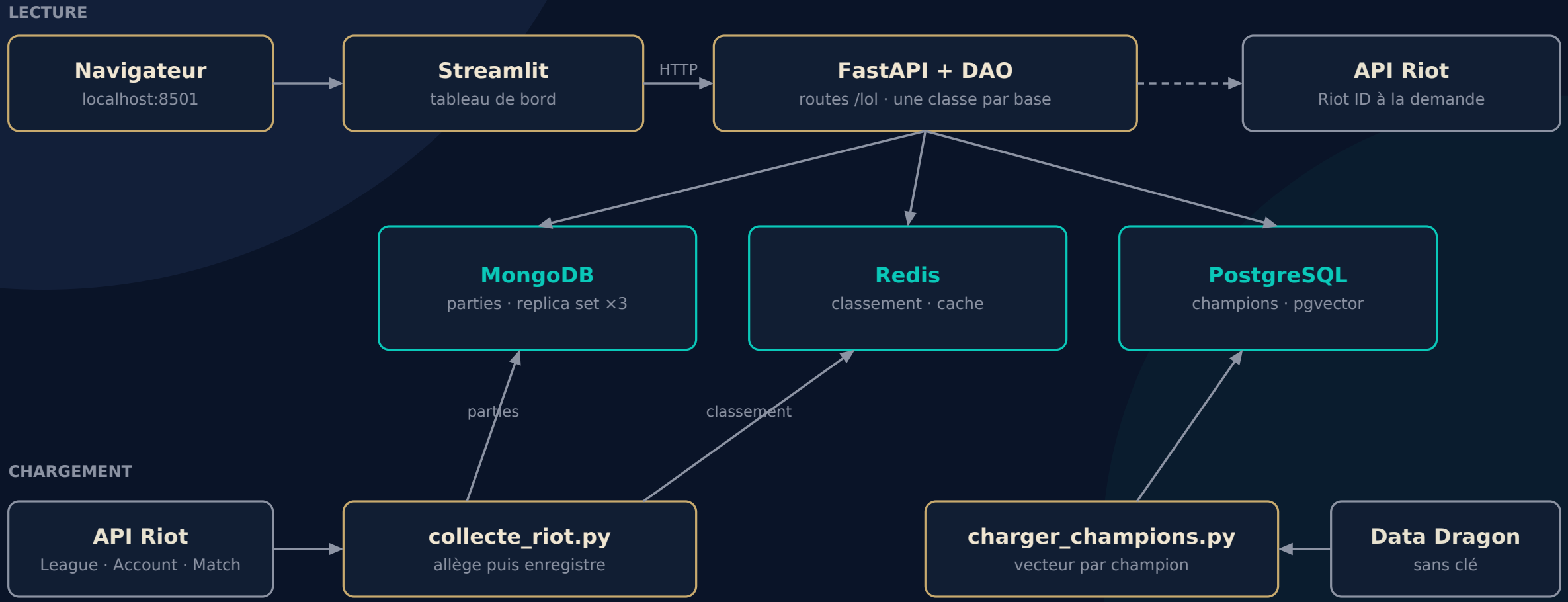
- Clé de développement gratuite, **valable 24 h**
- **100 requêtes toutes les 2 minutes** : une pause de 1,25 s entre deux appels
- Collecte complète : **574 appels, environ 15 minutes**

LICENCE ET CITATION

Données : **Riot Games**, API développeur (developer.riotgames.com) et Data Dragon (ddragon.leagueoflegends.com). Usage non commercial autorisé.

« Rift Stats isn't endorsed by Riot Games and doesn't reflect the views or opinions of Riot Games or anyone officially involved in producing or managing Riot Games properties. Riot Games, League of Legends are trademarks or registered trademarks of Riot Games, Inc. »

L'architecture



MongoDB : le modèle document

RIFTSTATS.PARTIES · UN DOCUMENT = UNE PARTIE

```
{ _id: 'EUW1_8036885434',      // matchId Riot
  date: ISODate('2026-11-05T18:12:21Z'),
  duree_s: 1615, patch: '16.22', file: 420,
  origine: 'ladder',           // ou 'recherche'
  participants: [              // les 10 joueurs
    { pseudo: 'PaIco#NA1', champion_id: 'Garen',
      role: 'TOP', equipe: 100, victoire: true,
      kills: 6, morts: 4, assists: 3,
      or: 11872, degats: 14671,
      objets: [6631, 3142, 3009, 3071, ...] },
    ... 9 autres ] }
```

- **Les 10 joueurs sont imbriqués** : ils sont toujours lus avec leur partie. Un document pèse **3,3 Ko en moyenne**.
- **13 champs gardés** sur plus de 1 000 renvoyés par Riot.
- **_id = l'identifiant Riot** et `replace_one(upsert=True)` : relancer la collecte ne crée **aucun doublon**.
- Les **parties annulées** (« remakes » de moins de 5 min) sont ignorées : 5 parties écartées.
- **origine** sépare la collecte des Challenger des joueurs cherchés dans l'interface.
- **3 index multiclés** sur des champs imbriqués : `participants.puuid`, `participants.champion_id`, `date`.

MongoDB : le taux de victoire

PIPELINE (SIMPLIFIÉ)

```
db.parties.aggregate([
  { $match: { origine: { $ne: 'recherche' } } },
  { $unwind: '$participants' },        // 1 ligne par joueur
  { $group: { _id: { champion: '$participants.champion_id',
                     role: '$participants.role' },
      parties: { $sum: 1 },
      victoires: { $sum: { $cond: ['$participants.victoire', 1, 0] } } } },
  { $sort: { parties: -1 } },          // rôle le plus joué d'abord
  { $group: { _id: '$_id.champion', role: { $first: '$_id.role' },
      parties: { $sum: '$parties' }, victoires: { $sum: '$victoires' } } },
  { $match: { parties: { $gte: 20 } } },
  { $project: { taux_victoire: { $round: [ 100 × victoires / parties, 1 ] } } },
  { $sort: { taux_victoire: -1 } } ])
```

Chaque résultat est gardé **60 s dans le cache Redis** : l'agrégation n'est recalculée qu'une fois par minute au plus.

RÉSULTAT · 510 PARTIES, 20 PARTIES MINIMUM

Champion	Rôle	Parties	Victoire
Kog'Maw	Haut	20	65,0 %
Aurelion Sol	Bas	34	64,7 %
Rell	Soutien	50	64,0 %
Zoé	Soutien	45	62,2 %
Ryze	Milieu	29	62,1 %
Lulu	Soutien	85	61,2 %

- **$bucket** : la victoire selon la durée de la partie. Ahri passe de 43 % (moins de 20 min) à 65 % (30 à 35 min).
- **$expr** : les matchups, c'est-à-dire l'adversaire de même rôle dans l'équipe d'en face.
- **Double $unwind** : les objets les plus achetés par champion.

Redis : classement et cache

Clé	Type	Commandes	Rôle
lol:classement	ZSET	ZADD · ZREVRANGE · ZREVRANK	300 joueurs triés par LP (score)
lol:joueur:<puuid>	HASH	HSET · HGETALL	pseudo, LP, victoires, défaites
lol:cache:*	STRING	SET … EX 60 · GET	réponses de l'API en JSON (cache-aside)
lol:rang:<puuid>	STRING	SET … EX 600	rang officiel d'un joueur cherché (10 min)

LE TOP 10 EN UNE COMMANDE

```
ZREVRANGE lol:classement 0 9 WITHSCORES
-- 1) Kaori#EUW33      4336
-- 2) Spear Shot#1111  4257
-- 3) Annie IRL#Annie  4123
ZREVRANK lol:classement <puuid>   -- 0 = premier
```

- **Pourquoi un ensemble trié ?** Le classement reste trié à chaque mise à jour, en O(log n). Le top 10 ou le rang d'un joueur s'obtiennent en une commande.
- **Cache-aside** : l'API lit d'abord Redis ; si la clé manque, elle calcule dans MongoDB puis écrit le résultat avec une durée de vie.
- **Petit volume** : 320 clés, 2 Mo de mémoire.

PostgreSQL + pgvector : la similarité

LE VECTEUR D'UN CHAMPION · 10 DIMENSIONS

0	0	1	0	0	0	.3	.4	.8	.5
Assas.	Comb.	Mage	Tireur	Sout.	Tank	Att.	Déf.	Magie	Diff.

Ahri : 6 classes (0 ou 1), puis 4 notes Data Dragon divisées par 10

LES 5 PLUS PROCHES (DISTANCE EUCLIDIENNE)

```
SELECT autre.nom,
       autre.embedding <-> ref.embedding AS distance
FROM lol_champions ref
JOIN lol_champions autre ON autre.champion_id <> ref.champion_id
WHERE ref.champion_id = 'Ahri'
ORDER BY distance LIMIT 5;
```

RÉSULTAT POUR AHRI

Champion	Distance
Aurora	0,00
Sylas	0,00
Kassadin	0,32
Katarina	0,35
Nidalee	0,37

- **Pourquoi PostgreSQL ?** Ce sont des données de référence, stables et bien structurées : elles ne changent qu'à chaque patch.
- Table `lol_objets` (863 objets) : elle donne le nom et le prix des objets trouvés dans les parties MongoDB.
- Au total, **224 ko** : tables, index et vecteurs.

8 · DÉMONSTRATION

L'application

Recherche

un champion, un joueur en base, ou n'importe quel Riot ID chargé depuis Riot

Champions

le top 10 avec ses barres, et tous les champions avec rôle, présence et KDA

Fiche champion

objets clés, victoire selon la durée, matchups, champions similaires

Joueurs

rang, victoire, KDA, champions favoris, 20 dernières parties

Classement

le podium et le top 10 des LP, lus dans Redis

Infra

état du replica set, taille des collections, mémoire Redis

DÉMO EN DIRECT · localhost:8501

- Chercher « **zoe** » : la recherche ignore les accents et ouvre la fiche de Zoé.
- Taper **notre propre Riot ID** (Pseudo#TAG), puis « Charger » : compte, rang et 15 parties arrivent en 3 à 5 s, puis le profil s'affiche.
- Ouvrir l'onglet **Infra** : le primaire et les deux secondaires.

CP ou AP ?

Usage	Que se passe-t-il si c'est un peu vieux ?	Notre choix
Taux de victoire	rien : 1 partie de plus ne change rien	AP : cache de 60 s accepté
Classement LP	un rang en retard de quelques minutes	AP : mis à jour à la collecte
Profil cherché	le rang affiché date de moins de 10 min	AP : cache de 10 min
Écriture d'une partie	une partie perdue fausserait les stats	CP : écriture majoritaire

MONGODB : REPLICA SET À 3 NŒUDS

- Les écritures vont au **primaire**. Elles sont confirmées quand **2 nœuds sur 3** les ont (`w: majority`, la valeur par défaut).
- Si le primaire tombe, les deux autres **élisent un nouveau primaire** en une dizaine de secondes. Pendant ce temps, on préfère **refuser l'écriture** plutôt que de la perdre.

NOTRE VERDICT

Une base CP, une lecture AP. Le stockage reste cohérent ; l'affichage accepte des données un peu en retard, servies par le cache Redis.

Démo : `docker stop mongo1`, puis « Rafraîchir » dans l'onglet Infra.

Limites et pistes

Limite	Constat	Piste
Échantillon	510 parties de 30 joueurs Challenger EUW. À 20 parties, l'intervalle de confiance à 95 % d'un taux de victoire fait ±21 points ; à 85 parties, ±10 points.	collecter davantage (Grandmaster, Master) et afficher l'intervalle de confiance
Clé Riot	Clé de développement valable 24 h, et 100 appels toutes les 2 min.	une clé « personal » ou « production » auprès de Riot
Panne de Redis	Les routes en cache échouent si Redis ne répond plus.	si Redis tombe, lire directement MongoDB, plus lentement mais sans erreur
Fraîcheur	La collecte se lance à la main.	une collecte planifiée, et une collection timeseries pour suivre l'évolution sur plusieurs patchs

Données personnelles : on ne garde que le Riot ID public et les statistiques de jeu. Aucun nom, aucun e-mail, aucune localisation.

Merci

Questions ?

- **MongoDB** pour les parties : documents imbriqués et agrégations
- **Redis** pour le classement et le cache : vitesse et données triées
- **PostgreSQL + pgvector** pour les champions : référentiel et similarité

Sources : Riot Games, API développeur et Data Dragon (patch 16.22.1). Rift Stats isn't endorsed by Riot Games.

POUR LES PRÉSENTATEURS · À NE PAS PROJETER

Annexe : déroulé de l'oral

Diapo	Sujet	Durée
1-2	Titre et besoin	1:00
3	Les données et le quota Riot	0:45
4	L'architecture	1:00
5-6	MongoDB : modèle et agrégation	2:00
7	Redis	0:45
8	PostgreSQL + pgvector	0:45
9	Démo en direct	2:00
10	CP ou AP	1:15
11-12	Limites, puis questions	0:30
	Total	10:00

AVANT L'ORAL (DANS LE DOSSIER NYC-EATS)

```
docker compose --profile tous up -d
docker exec -it --env-file riot.env python uvicorn api:app \
  --app-dir /projet --host 0.0.0.0 --port 8000 --reload
docker exec -it -w /projet python streamlit run \
  lol_tableau_de_bord.py --server.address 0.0.0.0
```

- **Régénérer la clé Riot** : elle expire 24 h après sa création. La recopier dans `riot.env`, puis relancer l'API.
- Ouvrir **localhost:8501** à l'avance : le badge doit être turquoise, « En direct ».
- Tester une fois la recherche de notre Riot ID, et prévoir une capture d'écran au cas où il n'y aurait pas de réseau.
- Après la démo CAP : `docker start mongo1`.
- Ne pas rendre `riot.env` : il contient la clé.